P2785R3
Relocating prvalues

Published Proposal, 2023-06-14

This version:
https://htmlpreview.github.io/?https://github.com/SebastienBini/cpp-relocation-proposal/blob/main/relocation.html

Authors:
Sébastien Bini (Stormshield)
Ed Catmur

Audience:
LEWG, EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes several mechanisms to enable real relocation in C++. We cover several topics, such as trivial relocatibility, container optimizations, supports for relocate-only types, library changes and the impact of the existing code base.

Table of Contents

§ 1. Overview

C++11 introduced rvalue references, move constructor and move-assignment operators. While these have improved C++ in many ways, there is still one operation that is unsupported: relocation. By "relocation", we mean the operation to construct a new object while simultaneously destructing the source object. Some other proposals look at this as "move + destruct", while we see this as a unique, different, and optionally better optimized operation.

This new operation brings several benefits, principally in terms of performance and code correctness. This proposal has its own take on trivial relocatibility (which allows to optimize the relocation operation into a simple `memcpy`). It also supports relocate-only types (such as `gsl::not_null<std::unique_ptr<int>>`) and enables to relocate constant objects.

The proposal introduces:

- two new special member functions: a relocation constructor `T(T)` and a relocation assignment operator `T& operator=(T)` ;
- a new keyword: `reloc` ;
- slight changes to overload resolution rules ;
- might introduce ABI breaks of some functions ;
- library evolution to support relocation.

The proposal does not introduce a new value type; instead relocation will happen from prvalues.

2. Motivation

2.1. Never-empty types

C++ lacks support for relocate-only types. Let's us consider the `gsl::not_null<std::unique_ptr<T>>` case for a moment. `gsl::not_null` inhibits the move constructor, as a moved-from pointer would be null and break the class invariant. In turn `std::unique_ptr` inhibits the copy constructor because of its ownership policy. Hence this type is non-copyable and non-movable. Those objects are legitimate since they improve code correctness (the user need not check whether they are empty) and performance (the compiler need not check whether they are empty).

Unfortunately those objects are quite impractical to handle in C++: they cannot be moved around in program memory (passed down to function, stored and removed from containers, etc...), while in principle there is no good reason to prevent that. With the relocation semantics in mind this would be allowed: each time the `gsl::not_null<std::unique_ptr<T>>` is moved in memory, it simultaneously destroys the previous instance (in practice, this simply means the memory occupied by the source object just becomes uninitialized, and in particular, the source object's destructor is not called).

2.2. Constant objects

Another place where C++ falls short is with constant objects. As of today, constant objects cannot be moved in C++ as the move constructor cannot steal the resources of a constant object. As such, never-modified objects that end their life by being moved (e.g. via `std::move`, or implicitly by return-value optimization where complete elision is not viable) cannot be marked as `const`. This is a missed opportunity and it leads to poorer code. The proposed relocation semantics solves this problem: constant objects can be relocated, they are just destroyed when done so.

Automatic objects are recommended by various guidelines to be constant, since a constant object will not change throughout its lifetime, simplifying reasoning about program state both for humans (coders, reviewers, maintainers) and for machines (compilers, tooling). Allowing an object to be `const` throughout its lifetime but give up its resources at the end of its lifetime allows for better, safer code.

As we will see other proposals about relocation provide little support for relocate-only types, and even less for constant objects. They only partly improve their support in a limited way.

2.3. Teachability

It is a matter of some confusion for learners that a "moved-from" object still exists and is accessible, but has an unspecified state. This leads to so-called "use-after-move" bugs, and requires static analysis passes and coding guidelines to prevent. "What does `std::move` do?" is, regrettably, a common "gotcha" question in a certain style of coding interview. In this

matter C++ compares unfavorably to other languages where the "move" operation is destructive (if trivial) and bars further access to the source object.

§ 2.4. Versus trivial relocation

As will be seen below, trivial relocation provides an incomplete solution to the issues presented in this section. The primary concern is that of composability; an aggregate of a self-referential type that cannot be trivially relocated (e.g. `std::string` in some implementations) and a relocate-only type (e.g. a non-null `unique_ptr`) can neither be relocated by move-and-destroy nor trivially relocated. Thus non-trivial relocation emerges as a requirement.

§ 2.5. Helper types

In framework code it is a relatively everyday task to write helper classes that are constructed to own some resource, passed around from place to place, and then destroyed, cleaning up the resource in the destructor. Move semantics for such classes requires identifying or adding an appropriate empty state, writing a move constructor, and adding checks to the destructor; all of these inhibit performance and are opportunities to introduce bugs. Relocation would replace all of this with a single defaulted special member function declaration.

§ 2.6. Library performance

Finally, relocation and especially trivial relocation will bring performance boosts in many situations. Other proposals make fine work at enumerating all the gains to be had from trivial relocation (see in particular [P1144R8]). To summarize, the performance gains are mainly in containers (std::vector::resize and the likes) and code size gains from functions that use `std::swap`.

§ 3. Nomenclature

We refer to the Terms and definitions of the C++ Standard, and to nomenclature introduced therein, in particular the object model. In addition, we define:

§ 3.1. Source and target objects

Relocation is the act of constructing a new instance while ending the lifetime of an existing one. This allows destructively stealing its resources, if any.

The new instance is called the *target object*. The existing instance whose lifetime is ended and whose resources are stolen is called the *source object*.

§ 3.2. Destructed state

An object is to be in a *destructed state* if its lifetime has ended because:

- its destructor was called, or ;
- it was passed to as source object to its relocation constructor.

It is a programming error to call the destructor of an object if it is already in a *destructed state*. As described in [basic.life], this has undefined behavior unless the object type is trivial, in which case its destructor or pseudo-destructor is a no-op.

3.3. Unowned parameter

An object is said to be an *unowned parameter* with regards to a function `f` if the object is a parameter of the function, passed by value, but the function `f` does not have control over its lifetime.

The lifetime of function parameters is implementation-defined in C++, but it is of most importance with relocation. Depending on the function call convention or ABI, the function may or may not be in charge of the lifetime of its parameters.

We denote two main parameter passing conventions:

- *caller-destroy*: the call site is in charge of the lifetime of the parameter passed in the function call ;
- *callee-destroy*: the function itself is in charge of the lifetime of its parameters ;

Depending on the ABI choice, the convention can be per parameter, or apply to all the function parameters. Other conventions may exist, and they are still compatible with this proposal.

For instance, in a function with caller-destroy convention, all its value parameters are *unowned parameters*. Likewise, with callee-destroy, none of its parameters are *unowned parameters*.

4. Comparison with existing proposals

This proposal introduces the `reloc` keyword, which allows users to explicitly and safely relocate local variables in their code base.

This proposal is also one of the few (with [P0308R0]), to tackle the case of relocate-only types. The `reloc` keyword thus becomes necessary to safely pass around such objects in user code.

Also, all these proposals (but [P0308R0]) aim to optimize the move and destruct operations into a single memcpy. But there are places where this optimization could not happen, and we are left with a suboptimized move and destruct. The relocation constructor that we propose offers a customization point, and especially allow for a more optimized relocation technique than move and destruct.

4.1. P1144R8: Object relocation in terms of move plus destroy by Arthur O'Dwyer

[P1144R8] puts the focus on trivially relocatable types, and offers ways to mark a type as trivially relocatable.

The current proposal has its own take on trivial relocatability that does not rely on a class attribute. Instead the trivial relocatability trait flows naturally from the new relocation constructor that we introduce. In short: if a class type has a relocation constructor that is (explicitly) declared as defaulted or is implicitly defined and not defaulted as deleted, and all of its subobjects are trivially relocatable (or fully trivial), then the type is trivially relocatable.

This approach is not prone to errors when compared to a class attribute, which could be mistakenly overriden by some users on classes that are not trivially relocatable.

Also, [P1144R8] does not account for relocate-only types. To some extend, trivially relocatable types have minimal support as they could be trivially relocated in the places where "move plus destroy" can be optimized. However:

- this only concerns trivially relocatable types ;
- there are places where the optimization cannot happen, and as such the object cannot be "relocated" ;
- this poses a composability issue. If a relocate-only (non-movable and non-copyable), trivially-relocatable type is a data member of a class that also has other non-trivially-relocatable data members, then the enclosing class becomes non trivially relocatable, while remaining relocate-only. This renders the enclosing class impractical to use without proper support for relocate-only types.

In this proposal we reuse [P1144R8]'s `std::relocate` function, except that we name it `std::destroy_relocate`.

4.2. P0023R0: Relocator: Efficiently moving objects by Denis Bider

The Relocator introduced in [P0023R0] is similar to the proposed relocation constructor. However P0023R0's Relocator is not viewed as a constructor. Instead, it is a special function that can be invoked in lieu of move plus destroy where possible.

However this brings again suboptimal support for relocate-only types. Indeed [P0023R0] does not force relocation to happen in all cases, and falls back to move+destroy paradigm when relocation cannot happen.

A typical example is when relocating a function parameter:

```
void sink(T);
void foo(T x) { sink(x); }
```

Here, under the terms of [P0023R0], relocation may not happen because of ABI constraints (if `x` is an unowned parameter). This will cause `foo` to fallback to a move+deferred detroy paradigm.

This proposal has another take on this issue: if `T` is relocate-only, then any function that takes a `T` parameter by value is required to have an ABI that allows it to relocate its input parameter (e.g. callee-destroy ABI).

This proposal also introduces the `reloc` keyword that is here to trigger the relocation, and protect against the reuse of the relocated object. The solution then becomes:

```
void sink(T);
void foo(T x) { sink(reloc x); /* x can no longer be used. */ }
```

Moreover, the proposed syntax for invoking [P0023R0]'s relocator is low-level and prone to error:

```
T x;
alignas(T) char buf[sizeof(T)];
T* y = new (buf) >>T(x);
```

Here the lifetime of `x` has been ended as if by a destructor call, but the language is not aware of this, so firstly the user may erreoneously refer to `x` as if it was within its lifetime, and secondly if an object is not recreated in `x` by the time the block exits, the behavior is undefined by [basic.life]/9. Likewise, the language is not aware of the existence of `*y` so will not call its destructor; the behavior is then possibly undefined under [basic.life]/5. In contrast, the present proposal would write the above as:

```
T x;
T y = reloc x;
```

Here the use of the `reloc` keyword ensures that the language is aware that the lifetime of `x` has finished, so the destructor only of `y` is called at scope exit.

4.3. N4158: Destructive Move by Pablo Halpern

[N4158] proposes a customizable function `std::uninitialized_destructive_move`, that is somewhat similar to the relocation constructor, but is a pure library solution.

It has several drawbacks :

- relocation can only happen if this function is called explicitly. Typically this function would be called in container implementation. But then we cannot relocate local variables with this.
- users can write their own `uninitialized_destructive_move` overload, but this is terrible for composability. Classes that have a subobject with a custom `uninitialized_destructive_move` overload do not get a `uninitialized_destructive_move` overload generated for free.

- `uninitialized_destructive_move` cannot be default-generated with memberwise relocation.

4.4. P1029R3: move = bitcopies by Niall Douglas

[P1029R3] enables a special bitcopies move constructor for types that are trivially relocatable.

Like the other proposals [P1029R3] does not provide support for relocate-only types (it suffers from the same drawbacks as [P1144R8] in that regard).

4.5. P0308R0: Valueless Variants Considered Harmful by Peter Dimov

We only consider the "pilfering" proposition from [P0308R0]. P0308R0's pilfering shares some similarities with the current proposal, as it is an attempt to support relocate-only types as a pure library solution.

We believe a language solution is best suited here:

- the source object is effectively destroyed by the relocation: its destructor is not called. This can hardly be achieved by a library solution ;
- the `reloc` keyword makes sure the relocated object is not reused, while `std::pilfer` does not ;
- the pilfering constructor is inconvenient to write as we need to unwrap from `std::pilfered` and rewrap to propagate to base classes and data-members ;
- as a library solution, the pilfering constructor cannot be defaulted ;
- trivial relocation is not possible with pilfering, which makes [P0308R0] miss the performance boost that is so longed for.

4.6. D2839R1: Nontrivial Relocation via a New *owning reference* Type by Brian Bi and Joshua Berne

[D2839R1] is very close in spirit and mechanism to this paper. We consider the addition of an "owning reference" to be an unnecessary complication to the type system that would impose a burden throughout library code. We refute the claim that a prvalue occupying storage constitutes a fourth value category; in our model, the lvalue that previously occupied that storage ceases to exist and a new prvalue is formed, which may come to occupy that storage (if relocation is elided) or distinct storage (in the general case).

We are intrigued by the claim that an owning reference can provided superior performance when passing a variable up the stack, compared to repeatedly relocating it from caller to callee, but believe that in practice such gains would be minimal and liable to be eliminated via optimization, while our model offers the possibility of improved locality, since the relocated object will reside directly in the leafmost function's stack as opposed to being accessible only via indirection.

Otherwise, the paper has some minor differences in naming (e.g. `std::force_relocate` for what we call `std::destroy_relocate`) that could easily be reconciled in either direction.

5. Proposed language changes

5.1. reloc operator

This paper suggests to introduce a new keyword, named `reloc`. `reloc` acts as a unary operator that can be applied to named, local, complete objects (in other words: function-local non-static variables and, with some restrictions, function parameters and identifiers introduced through the syntax of structured binding declarations).

`reloc obj` does the following:

- if `obj` is ref-qualified, then performs perfect-forwarding (as if by `static_cast<decltype(obj)>(obj)`);

- otherwise returns a temporary obtained from the source object, leaving it in a *destructed state* or a "*pending-destruction*" state ;
- in all cases, marks the "early" end-of-scope of the variable `obj`, preventing from use-after-relocation errors.

5.1.1. reloc to perfectly forward all value categories

`reloc` can be used on ref-qualified objects to enable perfect forwarding. If the source object is ref-qualified, then it performs the following cast: `static_cast<decltype(obj)>(obj)`.

This makes `reloc` the one operator to use to perfectly forward lvalues, xvalues and prvalues. It also prevents from use-after-move and use-after-relocation errors.

5.1.2. reloc returns a temporary

The main use case of `reloc` is to change the value category of an object to a prvalue. This happens by creating a temporary from the given source object, when the source object is not ref-qualified.

This temporary may or may not be materialized, depending on the context of the expression.

If materialized, then the temporary is initialized as follows:

- if the source object is not an *unowned parameter*, then the temporary is initialized using its relocation, move or copy constructor:
  - the relocation constructor is called if accessible. This is a destructive operation for the source object: when the constructor returns, the source object is considered to be in a *destructed state*.
  - otherwise, either the move or copy constructor is called, ignoring the cv-qualifiers on the source object. The destructor of the source object is then called at the end of the full-expression evaluation.
  - In all cases, the destructor of the source object is no longer called when its end of scope is reached.
- otherwise (the source object is an *unowned parameter*), then the temporary is initialized using its move or copy constructor, ignoring the cv-qualifiers on the source object. The destructor of the source object is deferred until the function returns. Typically we expect the source object to be in a "moved-from" state, waiting to be destructed.

If the temporary is not materialized and that the source object is not an *unowned parameter*, then the destructor of the source object is called at the end of the full-expression evaluation, at the same time as temporary objects are destroyed. See Discarded reloc expressions for more details on unmaterialized temporaries from `reloc` statements.

Note that an object passed to `reloc` is guaranteed to be either in a *destructed state* at the end of the expression evaluation if it is not an *unowned parameter*, or else in a "*pending-destruction*" state.

5.1.3. Illegal uses of reloc

A `reloc` statement is ill-formed if any of the following conditions is met:

- its parameter is not the name of a variable ;
- the source object is not a *complete object* ;
- the source object is not ref-qualified and does not have local storage (i.e. not a local function variable and not a function parameter passed by value) ;
- the source object is not ref-qualified and does not have an accessible relocation, move, or copy constructor ;
- the source object is a structured binding (and not an identifier introduced in a structured relocation declaration) ;
- the source object is a lambda capture from a non-decomposing lambda ;

In particular, the `reloc` statement is well-formed if the source object is a member of a function-local anonymous union.

For instance:

```
void foo(std::string str);
std::string get_string();
std::pair<std::string, std::string> get_strings();

std::string gStr = "static string";

void bar(void)
{
        std::string str = "test string";
        foo(reloc str); // OK: relocation will happen given that std::string has a reloc ct
        foo(reloc gStr); // ill-formed: gStr does not have local storage

        std::pair p{std::string{}, std::string{}};
        foo(reloc p.first); // ill-formed: p.first is not a complete object, and not the na

        foo(reloc get_string()); // ill-formed: not the name of variable
        foo(reloc get_strings().first); // ill-formed: not a complete object, and not the n
}

void foobar(const std::string& str)
{
        foo(reloc str); // OK: str is passed by reference.
        // Note that the lifetime of the referent of str is unaffected.
}
void foobar(std::string* str)
{
        foo(reloc *str); // ill-formed: *str is not the name of variable
}
void foobar2(std::string* str)
{
        foobar(reloc str); // OK, the pointer itself is relocated (not the pointed value)
}

class A
{
        std::string _str;
public:
        void bar()
        {
                foo(reloc _str); // ill-formed: _str is not a complete object and does not
        }
};
```

5.1.4. Early end-of-scope

`reloc obj` simulates an early end-of-scope of `obj`. It does so by forbidding any further mention of the name `obj` which would resolve into the relocated object.

If `obj` is a member of an anonymous union, then `reloc` does not prevent name reuse; all members of that union, including the one that was relocated, can be reused normally.

Pointers and references that pointed to `obj` become dangling, and we don't try to offer any protection against that. We only protect against reusing the variable that was passed to `reloc`.

The program is ill-formed if `reloc obj` is used in an instruction and any of the following is true:

- in at least one code path from after the instruction that contained `reloc obj` up to the natural end-of-scope of said `obj`, the name `obj` is reused and it resolves to the object that was passed to `reloc` ;
- within the instruction where `reloc obj` is used, the name `obj` is reused in the same conditional branch.

The code path evaluation performed to detect such ill-formed programs is based only on compile-time evaluations, disregarding run-time values:

- For any non-constexpr `if` block encountered in the code path, the true branch must be considered as well as the else branch (if it exists).
- If an `if constexpr` block is encountered, only the branch that evaluates to true is considered.
- Any `for` or `while` loop body is considered to be entered once if `reloc` is used before the `for`/`while` block or in the init-statement of the `for` loop.
- If `reloc` is used within a loop body, a loop condition, or a for-loop iteration expression, and that there exists a code path from the `reloc` instruction up to the next iteration of the loop (i.e. no `return` statement, no `break` statement that applies to that loop, or no `goto` statement that jumps after that loop) then the loop is considered to happen one more time for code-path evaluation.

Consider the following examples:

```
void relocate_case_01()
{
        const T var = getT();
        bar(reloc var);
        if (sometest(var)) // ERROR
                do_smth(var); // ERROR
}
```

`var` cannot be reused after the `reloc` call.

```
void relocate_case_02()
{
        const T var;
        {
                const T var;
                bar(reloc var);
                do_smth(var); // ERROR, var cannot be reused after the <code data-opaque bs
                {
                        const T var;
                        do_smth(var); // OK
                }
                do_smth(var); // ERROR, var cannot be reused after the <code data-opaque bs
        }
        do_smth(var); // OK
}
```

The second and forth calls to `do_smth(var)` are allowed because the name `var` does not resolve to the relocated object.

```
void relocate_case_03()
{
        const T var = getT();
        if (sometest(var))
                bar(reloc var);
        else
                do_smth(var); // OK
}
```

`do_smth(var)` is allowed because the `else` branch is not affected by the `reloc` call of the `if` branch.

```
void relocate_case_04()
{
        const T var = getT();
        if (sometest(var))
                bar(reloc var);
        else
                do_smth(var); // OK
        // [...]
        do_smth_else(var); // ERROR
}
```

`do_smth_else(var)` is an error because `var` is mentioned after the `reloc` call.

```
void relocate_case_05()
{
        const T var = getT();
        if (sometest(var))
                bar(reloc var);
        else
                do_smth(reloc var); // OK
}
```

Both `reloc` are okay as they happen on different code paths.

```
void relocate_case_06()
{
        const T var = getT();
        bool relocated = false;
        if (sometest(var))
        {
                bar(reloc var);
                relocated = true;
        }
        else
                do_smth(var); // OK
        // [...]
        if (!relocated)
                do_smth_else(var); // ERROR
}
```

It does not matter that the developer attempted to do the safe thing with the `relocated` variable. The code-path analysis disregards run-time values and explores all branches of an `if` block (unless in the case of `if constexpr`).

```
void relocate_case_07()
{
        constexpr bool relocated = my_can_relocate<T>{}();
        const T var = getT();
        if constexpr(relocated)
        {
                bar(reloc var);
        }
        else
                do_smth(var); // OK
        // [...]
        if constexpr(!relocated)
                do_smth_else(var); // OK
}
```

The above example is safe because of the use of `if constexpr`.

```
void relocate_case_08()
{
        const T var = getT();
        if (sometest(var))
        {
                bar(reloc var);
                return;
        }
        do_smth(var); // OK
}
```

This example is also safe thanks to the `return` statement right after the `reloc` instruction, which prevents from running `do_smth(var);`.

```
void relocate_case_09()
{
        const T var = getT();
        for (int i = 0; i != 10; ++i)
                do_smth(reloc var); // ERROR
}
```

This is ill-formed as each iteration reuses `var` which is declared before the loop body. Even if `i` were compared against `1` or even `0` (for respectively one iteration, or no iteration) then the program would still be ill-formed. Run-time values (like `i`) are disregarded in the code-path analysis that comes with `reloc`. The analysis will report that there is an optional code jump, after the `do_smth` call (and `reloc var`), which jumps to before the `reloc var` call and after the initialization of `var`. Although the jump is optional (depends on `i`, whose value is disregarded for this analysis) it may still happen and thus such code is ill-formed.

```
void relocate_case_10()
{
        const T var = getT();
        for (int i = 0; i != 10; ++i)
        {
                if (i == 9)
                        do_smth(reloc var); // ERROR
                else
                        do_smth(var); // ERROR
        }
}
```

This is ill-formed for the same reason as above. The code-path analysis will report that any iteration of the for-loop may take any branch of the if statement and potentially reuse a relocated variable.

```
void relocate_case_11()
{
        const T var = getT();
        for (int i = 0; i != 10; ++i)
        {
                if (i == 9) {
                        do_smth(reloc var); // OK
                        break;
                }
                else
                        do_smth(var); // OK
        }
}
```

Adding the break statement right after the `reloc` call makes the code snippet well-formed. Indeed the `break` statement forces the loop exit, which implies that the conditional jump at the end of loop (that may start the next iteration) is no longer part of the code path that follows the `reloc` instruction.

```
void relocate_case_12()
{
        for (int i = 0; i != 10; ++i)
        {
                const T var = getT();
                do_smth(reloc var); // OK
        }
}
```

`var` is local to the for-loop body, so `reloc` is well-formed here. The next loop iteration inspected by the code path analysis will see that `var` is a new object that shadows that of the previous iteration.

```
void relocate_case_13()
{
        const T var = getT();
from:
        if (sometest(var)) // ERROR
        {
                do_smth(var); // ERROR
        }
        else
        {
                do_smth(reloc var);
        }
        goto from;
}
```

Because of the `goto` instruction, `var` may be reused after `reloc var`.

```
void relocate_case_14()
{
        const T var = getT();
from:
        if (sometest(var)) // OK
        {
                do_smth(var); // OK
                goto from;
        }
        else
        {
                do_smth(reloc var);
        }
}
```

In this scenario `goto` is placed in a way that does not trigger the reuse of relocated `var`.

```
void relocate_case_15()
{
        union { T x; U y; };
        x = getT();
        if (sometest(x))
        {
                do_smth(reloc x);
                x = getT(); // well-formed
        }
        else
```

```
        {
                reloc x; // destroy x
                y = getU();
                do_smth_else(reloc y); // well-formed
                x = getT(); // well-formed
        }
        do_smth_else(reloc y); // well-formed
}
```

`reloc` does not prevent reusing the names of anonymous union members.

5.1.5. Conditional relocation

It may happen that the `reloc` is invoked in the some code branches and not in others:

```
void foo()
{
    T obj = /* ... */;
    if (some_test())
        bar(reloc obj);
    else
        handle_error();
    live_on();
}
```

This code is well-formed. `obj` is relocated in the `if` branch (left in a *destructed state*), and not in the `else` branch. When `obj` reaches its end of scope, the function needs to know whether `obj` is in a *destructed state* in order to elide its destructor call.

This requires extra tracking, which will probably take the form of extra flags in the function stack. Given a source object, this tracking is only necessary if all the following conditions are met:

- the source object is not an *unowned parameter* ;
- the source object is not trivial (its destructor is not a no-op) ;
- in conditional branches, `reloc` is used in at least one branch and not used in at least one branch as well (taking into account the potentially empty `else` branch).

We prefer to leave the details of this tracking implementation-defined.

Concerns have been expressed that extra flags would violate the "don’t pay for what you don’t use" principle. We acknowledge this concern, but feel that the user is getting at least something out of this. Alternatives would be for the source object to be destroyed implicitly at the opening or closing brace of the `else`, for the code to be ill-formed unless the source object is relocated within the `else`, or for the code to be ill-formed unconditionally. Note that in each of these cases, the user can recover our preferred behavior in library, using `std::optional` to add the tracking flags that in our proposal would be provided by the language.

5.1.6. Relocation elision

Whether performed by relocation, move or copy constructor, relocation may be elided if the compiler can ensure that the source object is created at the address to be occuped by the target object. This is intended to work in much the same way as the named return value optimization; for example:

```
void f(std::string s);
void g() {
    std::string s;  // may be created in f’s argument slot
```

```
    f(reloc s);  // relocation may be elided
}
```

5.1.7. Discarded reloc expressions

Under some conditions the following expression is well-formed: `reloc obj;` (note the semi-colon).

Given the rules we established for `reloc`, this statement returns a temporary constructed from the source object. The temporary is then destructed at the end of the expression evaluation.

However, materializing a temporary whose only goal is to be destroyed is suboptimal. Hence, it is authorized for implementations to (as above) elide the creation of the temporary object, effectively only calling the destructor of the source object at the end of the expression evaluation. If this optimization is done, the temporary returned by `reloc` is not materialized.

This means that `reloc obj;` has the following behavior:

- if `obj` is not an *unowned parameter* then:
  - The temporary is likely elided, effectively only calling the destructor of `obj`.
  - Otherwise a temporary is initialized from the source object, and then destructed.
- otherwise (`obj` is an *unowned parameter*):
  - A temporary is created from the move or copy constructor, and then destructed.
  - The source object destruction is deferred until the function returns.

In particular, `reloc obj;` is ill-formed if the source object has no accessible relocation, move or copy constructor.

For instance this gives:

```
void do_something_01(std::mutex& m)
{
    std::lock_guard guard{m};
    if (!some_test())
    {
        reloc guard; // ill-formed: no relocation or move constructor
        log("thread ", std::this_thread::get_id(), " failed");
        return;
    }
    bar();
}
void do_something_02(std::unique_lock<std::mutex> guard)
{
    if (!some_test())
    {
        reloc guard; /* well-formed: lock is released, either by calling the
            destructor directly, or by constructing a temporary from
            guard (by relocation or move) and destructing it. */
        log("thread ", std::this_thread::get_id(), " failed");
        return;
    }
    bar();
}
void do_something_03(std::mutex& m)
{
    std::unique_lock guard{m};
    if (!some_test())
    {
```

```
        reloc guard; /* well-formed: temporary is likely elided regardless of
            do_something_03's ABI, only calling the destructor of guard. */
        log("thread ", std::this_thread::get_id(), " failed");
        return;
    }
    bar();
    /* guard destructor is called only if it wasn't passed to reloc. */
}
```

5.2. Relocation constructor

We introduce the relocation constructor. As relocation happens from prvalues, the constructor takes a prvalue as parameter: `T(T)`.

This signature was picked as it completes the C++ tripartite value system. The copy constructor creates a new instance from an lvalue, the move constructor from an xvalue, and then the relocation constructor from a prvalue.

> NOTE: a further benefit of this syntax is that it is currently ill-formed [class.copy.ctor]/5, and thus available for extension.

A point of confusion may be that the syntax implies an infinite regress: the parameter must be constructed, which requires a prior call to the relocation constructor, and so on. This is not the case; if the source object was previously a glvalue the operand of the `reloc` operator, it was transformed into a prvalue immediately before entering the relocation constructor, and the parameter of the relocation constructor is that same prvalue. (If the source object was already a prvalue, there is no issue; the parameter is that prvalue.)

An attractive intuition is that the parameter aliases the source object in the same way as a reference or a structured binding declaration. However, this is misleading; the lifetime of a source object glvalue has already ended and so use of a pointer or reference referring to the source object has undefined behavior, except as provided by [basic.life] and [class.cdtor].

> NOTE: this behavior matches that for the destructor of a class type; see [basic.life] paragraph 1.

This intuition is only useful in so far as the ABI for a relocation constructor prvalue parameter is likely to be the same as that for a copy or move constructor parameter, since the prvalue parameter may have the same storage location as a previously existing glvalue.

> NOTE: it does not matter that the ABI for the relocation constructor parameter differs from that for a prvalue parameter in normal functions, since it is not possible to take the address of a constructor.

The role of the relocation constructor is to construct a new instance by destructively stealing the resources from the source object. Unlike the move constructor, the relocation constructor needs not to leave the source object in valid state. In fact the lifetime of the source object was ended immediately prior to entering the relocation constructor, and thus the source object must simply be considered as uninitialized memory after the relocation constructor terminates. We also say that the source object is left in a *destructed state*. This means that the destructor of the source object must no longer be called (and will not be called, assuming the `reloc` operator was used).

5.2.1. Declaration

The relocation constructor can be declared (implicitly or explicitly), defaulted and deleted like any other constructor.

The relocation constructor of a class-type `T` implicitly gets a `noexcept(true)` exception specification unless:

- it is explicitly declared with `noexcept(false)` ;
- or one `T`'s subobjects has a `noexcept(false)` relocation constructor ;

- or one `T`'s subobjects does not declare a relocation constructor and has a `noexcept(false)` move constructor.

These rules are similar to that of the destructor's implicit exception specification.

A class-type that provides a relocation constructor has some impact on the program ABI. See the ABI section.

§ *5.2.1.1. Implicit declaration*

If a class-type follows the Rule of Zero (updated to account for the relocation constructor and relocation assignment operator), then the compiler will declare a non-explicit inline public relocation constructor, i.e. if none of the following are user-declared:

- copy constructor
- copy assignment operator
- move constructor
- move assignment operator
- destructor
- relocating assignment operator

§ *5.2.1.2. Deleted implicitly-declared or defaulted relocation constructor*

The implicitly-declared or defaulted relocation constructor for class `T` is defined as deleted:

- if `T` has subobjects that explicitly declare a deleted relocation constructor ;
- or `T` has subobjects with missing relocation and move constructors (i.e. that are deleted, inaccessible, or ambiguous).
- or `T` has subobjects with deleted or unaccessible destructor.

As for move constructors, a defaulted relocation constructor that is deleted is ignored by overload resolution.

> NOTE: this means that a class with an explictly deleted relocation constructor will still be relocated if necessary, but through the move (or copy) constructor and destructor.

§ *5.2.1.3. Trivial relocation*

A relocation constructor of a class type `X` is *trivial* if it is not user-provided and if:

- `X` has no virtual functions and no virtual base classes, and
- for each direct base class and direct non-static data member of class type or array thereof, the relocation operation (which may be a relocation constructor or synthesized from copy/move constructor plus destructor) selected to relocate that base or member is trivial.

A *trivially relocatable class* is one which:

- has a trivial, eligible relocation constructor, or
- does not have a relocation constructor (including one that is deleted), and is trivially copyable.

> NOTE: *eligible* is defined in [special].

Scalar types, trivially relocatable class types, and arrays and cv-qualified versions thereof are *trivially relocatable*.

> **we also tighten the definition of "trivial class" (and thus "trivial") to require that the class in question be trivially relocatable as well as trivially copyable. This is to ensure that if the user wants code to be called on relocation, the library does not bypass said code by, say, using `memmove`.**

5.2.2. Definition

5.2.2.1. Default definition

The default relocation constructor implementation for a class-type `T` depends on `T`'s type traits.

If `T` is trivially relocatable then the relocation constructor effectively (ignoring padding) performs a memcpy over its entire memory layout.

Otherwise in the nominal case, the constructor implementation performs memberwise relocations.

In the relocation constructor `T(T src)`, for each subobject `s` (of type `S`) of `T`, in declaration order:

- if `S` has an accessible relocation constructor, then `this->s` is constructed by calling the relocation constructor, passing `src.s` as source object. `src.s` is then left in a *destructed state*. Note that if `S` is trivially relocatable then the relocation constructor call may be optiomized into a memcpy: `memcpy(&this->s, &src.s, sizeof(S))` ;
- otherwise if `S` has an accessible move constructor and destructor, then `this->s` is constructed by calling the move constructor, ignoring `src.s`'s cv-qualifiers. This operation is called *synthesized relocation*. `src.s` is **not** in a *destructed state* ;

When all target subobjects have been constructed, the destructors all of source subobjects are called, in reversed declaration order, omitting those that are already in a *destructed state*.

5.2.2.2. User-provided definition

Users can provide their own definition of the relocation constructor. Special rules apply to the relocation constructor's member initialization list: subobjects that have no user-provided initialization will be constructed by relocation or synthesized relocation, instead of being default-constructed.

In other terms, subobjects that are omitted in the member initializer list are not constructed using their default constructor, but instead are constructed using relocation from the matching source subobject. That relocation is performed either by the relocation constructor or by synthesized relocation, using the rules described in the default relocation constructor implementation. If synthesized relocation happened for a subobject, then the source subobject is not in a *destructed state* yet.

Before the relocation constructor body is entered, the destructors of all the source subobjects are called in reversed declaration order, omitting those that are in a *destructed state*. In particular a source subobject is not in a *destructed state* if the target subobject has a user-provided initialization in the member initialization list, hence the destructor of such subobject is always called before the constructor body is entered.

At the end of the member initialization list, the whole source object is left in a *destructed state*. Using the source object in the constructor body leads to an undefined behavior.

Consider the following examples:

```
struct T
{
    std::string _a, _b, _c;

    T(T src) :
        _a{std::move(src._a)}, _b{} {} /*
        1. T::_a is constructed using the move constructor.
        2. T::_b is default constructed.
```

```
        3. T::_c is constructed using std::string's relocation constructor, from src._c.
        4. src._b and src._a are destructed (in that order) before the constructor body is
    */
};

struct U
{
    std::string _a, _b;
    U(U src) {} /*
        U relocation constructor behaves like the default definition, although it
        counts as user-provided.
    */
};

class List
{
public:
    List(List src) /* _sentinel is memcpied from src._sentinel */
    {
        /* fixup references */
        _sentinel._prev->_next = &_sentinel;
        _sentinel._next->_prev = &_sentinel;
    }
private:
    struct Node { Node* _prev; Node* _next; int _value; };
    Node _sentinel;
};
```

Alternatively, if the user calls a delegating constructor in place of a member initializer list, then the destructor of the source object is called right after the delegating constructor call completes.

This further means that the source object is fully destructed by the time the relocation constructor body is entered. Any operation on it may result in undefined behavior. However the source object name can still be accessed for debugging purposes (like printing its address somewhere). Compilers can still emit warnings when undefined uses of the source object are done in the constructor body.

It is not possible for users to explicitly call the relocation constructor on subobjects. This is because there is no existing syntax to do so:

- `T(T src) : _a{src._a}` calls the copy constructor.
- `T(T src) : _a{auto{src._a}}` calls the copy constructor to make a temporary, which then gets passed to the relocation constructor.
- `T(T src) : _a{reloc src._a}` is not permitted as we cannot call `reloc` on a subobject.
- `T(T src) : _a{std::destroy_relocate(&src._a)}` is erroneous as well, as `src._a` will be destructed at the end of the member initializer list.

This is the reason why omitted subobjects are automatically constructed by relocation, and not using their default constructor. If users want to default-construct some subobject, then they can write it explicitly: `T(T src) : _a{} {}` (in which case the source subobject is destroyed at the end of the initializer list).

It is for safety reasons that the relocation constructor ensures that the source object is entirely destroyed by the time the constructor's body is reached. Had it been otherwise, then it would have been the responsibility of the users to destroy the subobjects that did not get relocated. This would likely lead to programming errors, especially when we consider synthesized relocation.

5.2.2.3. Exception handling

The relocation constructor is able to handle exceptions. If an exception leaks through the relocation constructor then it guarantees that the *target* is not constructed and the *source* object is destroyed.

This is in general undesirable, which is why the relocation constructor is `noexcept` if at all possible.

As we have seen above, the relocation constructor acts in three stages: (a) target subobjects construction, (b) destruction in reversed declaration order on any source subobject that is not in a *destructed state* (because of synthesized relocation or user-provided initialization), (c) the function body.

Stage A: target subobjects construction

If an exception leaks through in stage (a) then:

1. in reversed declaration order, call the destructor of all initialized subobjects.
2. in reversed declaration order, call the destructor of the source subobjects that are not in a *destructed state*:
    - all subobjects whose corresponding target subobject *didn't* get initialized ;
    - all subobjects whose corresponding target subobject *did* get initialized, but through synthesized relocation or user-provided initialization ;
    - if the initialization that threw did not happen through a relocation constructor call, then the matching subobject. (If the initialization happened by relocation then we know that the source subobject is in a *destructed state*.)

We call the destructor on the target subobjects first as they were constructed more recently.

Stage B: source subobjects destruction

If an exception leaks through in stage (b) then:

- All target subobjects are destroyed in reversed declaration order.
- All the remaining destructors of the source subobjects are called in reversed declaration order.

Stage C: constructor body

If an exception leaks through in stage (c) then all target subobjects are destroyed in reversed declaration order, like it is the case for any constructor.

Delegating constructor case

If an exception leaks through the delegating constructor then the source object is destructor is called and the exception is propagated.

Note that the target object needs not to be destroyed as the delegating constructor already took care of that.

5.2.2.4. Additional parameters

As with copy and move constructors, it is permissible to add additional parameters to a relocation constructor, on condition they have a default initializer.

One case where this can be of use is if the user needs space to store information and/or resources for the duration of the relocation constructor, for a contrived example:

```
class T
{
public:
    class Helper {
    public:
```

```
        Helper() = default;
        ~Helper() { delete p; }
    private:
        friend T;
        int* p;
    };

    T(T src, Helper storage = {}) noexcept(false)
        : _p(storage.p = std::exchange(src._p, nullptr))
    {
        storage.p = nullptr;
    }

    ~T() {
        delete _p;
    }

private:
    int* _p;
    RelocateOnly _q;
    ThrowingRelocate _r;
};
```

In the above, `T::_p` does not manage its own lifetime, but the presence of `T::_r` means that `T::T(T)` is not noexcept so we need to release its resources if an exception is thrown during relocation. The presence of `T::_q` demonstrates that relocation cannot be synthesized.

5.2.3. Invocation

The relocation constructor is invoked *as necessary* to relocate a prvalue from one storage location to another. Use of the `reloc` operator does not guarantee that a relocation constructor (if present) will be called, since it may be elided if the compiler can arrange that the source glvalue was constructed at the appropriate address.

In particular, code of the form `T x = reloc y;` is *highly* likely to be a no-op, simply renaming an existing object. This is however likely to find use for "sealing" objects with complex initialization, replacing the idiom of immediately-invoked function expressions (IIFEs, [IIFE]):

Before	After

`T const x = std::invoke([&]`
`{`
`  T x;`
`  x.modify(y, z);`
`  return x;`
`});`
	`T x_mut;`
	`x_mut.modify(y, z);`
	`T const x = reloc x_mut;`

Or, consider:

```
C f(int i) {
    C c1, c2;
    if (i == 0)
        [[likely]] return reloc c1;  // #1
    else if (i == 1)
        [[likely]] return c1;  // #2
    else
        [[unlikely]] return c2;  // #3
}
```

At `#1` the `reloc` is largely redundant; the end-of-life optimization means the compiler is entitled to treat `c1` as a prvalue anyway, as in `#2`. Indeed, the likelihood annotation encourages the compiler to construct `c1` in the return slot, such that both `#1` and `#2` are a no-op. It is only `#3` that is likely to invoke the relocation constructor.

The relocation constructor may also be invoked by library functions, for example § 6.1.2 std::destroy_relocate.

§ 5.3. Relocation assignment operator

We further introduce the relocation assignment operator. Its signature shall be: `T& T::operator=(T)`. Such operators may already be defined in existing codebases, but the proposed changes will not interfere with them.

Sometimes we also make mentions to the *prvalue-assignment operator*. It refers to the same function, but further indicates that this function existed prior to the proposal.

§ 5.3.1. Declaration

The relocation assignment operator becomes a special member function. As such, declaring one breaks the Rule of Zero, which was not the case previously.

The relocation assignment operator may be implicitly declared, and may be defaulted or deleted.

§ *5.3.1.1. Implicit declaration*

If a class-type follows the Rule of Zero, then the compiler will declare an inline public relocation assignment operator.

§ *5.3.1.2. Deleted implicitly-declared or defaulted relocation assignment operator*

The implicitly-declared or defaulted relocation assignment operator for class `T` is defined as deleted:

- if `T` has subobjects that have an implicitly or explicitly deleted relocation assignment operator ;
- or `T` has no relocation, move, or copy constructor ;
- or `T` has subobjects that have inaccessible relocation or move assignment operators ;
- or `T` has subobjects with deleted or unaccessible destructor.

A defaulted relocation assignment operator that is deleted is ignored by overload resolution.

§ 5.3.2. Relocation assignment operator parameter relocation elision

As with the relocation constructor, it is desirable that the parameter should be the source object *converted to* a prvalue, and not a temporary prvalue relocated *from* the source object. This is particularly critical for the default definition of the operator, which (as you might suspect) performs memberwise calls to other relocation assignment operators. Without elision, that would imply recursive relocation of each subobject, down to their smallest unbreakable parts.

This, however, poses a problem, since it is possible to take the address of a relocation assignment operator, yielding a pointer (or reference) with (typical) signature `T& (T::)(T)`, implying that the source object must occupy a parameter slot, which may not find it possible to have the same storage address as the source object, and/or which the caller may expect to destroy (see § 5.7 ABI changes).

Nevertheless, we mandate elision where possible:

- If the class-type (possibly implicitly) declares a non-deleted relocation constructor, or declares a defaulted relocation assignment operator, then elision is mandated at declaration level ;

- Otherwise, if the class-type defines a relocation assignment operator as defaulted, then elision is mandated at definition level ;
- Otherwise elision is not mandated.

Elision is performed in such a way as to avoid ABI break (more on that on the ABI section).

5.3.2.1. Elision at declaration level

If elision is mandated at declaration level, then the assignment operator declaration actually declares two member functions:

- the non-eliding one, which takes its input parameter by value. This is the function that will get called when user-code calls the assignment operator. It is the prvalue-assignment operator as we know it today ;
- the eliding one, which takes its input parameter as if by reference, and has the same return type as the non-eliding one. The eliding function has no identifier and does not participate in overload resolution. Users cannot take its address and this function cannot be called directly in user-code. The eliding operator is in charge of destructing its source object, by relocation or destructor call.

The definition of the assignment operator (which is user-provided or defaulted) will serve as the definition of the eliding operator.

The non-eliding operator definition is generated by the compiler, and merely wraps the call to the eliding one:

- If the source object passed to the non-eliding operator is not an *unowned parameter*, then the operator:
  1. Calls the eliding operator, passing the source object as if by reference.
  2. It forwards as return value whatever the eliding operator returns.
  3. Upon function exit, it does not destroy the source object as it is already in a *destructed state*.
- Otherwise (the source object is an *unowned parameter*), then the operator:
  1. Creates a copy of the source object, using move or copy constructor.
  2. Calls the eliding operator, passing that copy by reference.
  3. It forwards as return value whatever the eliding operator returns.
  4. Upon function exit, it does not destroy the copy as it is already in a *destructed state*.

If the address of the assignment operator is queried, then the address of the non-eliding version is returned. If the assignment operator is virtual, then only the non-eliding version is considered to be `virtual` and is added to the vtable entry.

5.3.2.2. Elision at definition level

If elision is mandated at definition level, then the two versions of the operator are generated (eliding and non-eliding) in the translation unit where the operator is defined. The visibility of the eliding operator symbol to other translation units is implementation-defined.

The definition of the two functions are the same as if elision was mandated at declaration level.

5.3.3. Definition

5.3.3.1. Default definition

The default definition of the operator, given the rules above, benefits from elision. In particular, the default definition is responsible for the destruction of its source object.

As you would expect, the default definition merely delegates to the relocation assignment operator of all its subobjects.

In `T`'s default assignment operator, for all subobjects `s` of `T` of type `S`:

- If `S`'s relocation assignment operator mandates relocation elision at declaration level, then call it with `src.s` as parameter (passed as if by reference thanks to elision). The source subobject is then left in a *destructed state*.
- Otherwise if `S` provides an non-eliding relocation assignment operator but has an accessible relocation constructor, then call the operator, if necessary relocating `src.s` into the parameter slot. The source subobject is then left in a *destructed state*.
- Otherwise if `S` provides an non-eliding relocation assignment operator and no accessible relocation constructor, then call the operator by passing a temporary copy of `src.s`. This temporary is move-or-copy-constructed, ignoring the potential cv-qualifiers on `s`. The source subobject is *not* destructed.
- Otherwise if `S` provides a move assignment or copy assignment operator, then call the operator: `this->s.operator=(std::move(src.s));`. The source subobject is *not* destructed.

After all the assignment operator calls have been made, the destructors of all source subobjects that are not in a *destructed state* are called in reversed declaration order.

This subobject destruction phase also happens as-is during stack-unwinding if one of the assignment operators throws, effectively ensuring that the source object will be left in a *destructed state*.

§ *5.3.3.2. Possible user definitions*

Unlike the relocation constructor, the relocation assignment operator does not rely on some special member initialization list. Instead, the assignment operator relies on existing mechanisms.

The two patterns commonly used to implement the assignment operator still work as expected.

relocate-and-swap

```
T& operator=(T src)
{
    swap(*this, src);
    return *this;
    /* src destructor will still be called using normal rules */
}
```

destroy-and-construct

```
constexpr T& operator=(T src) noexcept
{
    static_assert(std::is_nothrow_destructible_v<T> &&
        std::is_nothrow_relocatable_v<T>);

    std::destroy_at(this);
    return *std::construct_at(this, reloc src);
}
```

Let’s have a look at what happens in this function:

- The static assertion makes sure we have a relocation constructor. Hence, the relocation assignment operator comes with an eliding and a non-eliding version ;
- The provided definition will be used for the eliding version of the operator ;
- As its relocation is elided, `src` is not an unowned parameter. As a consequence, `reloc src` will effectively call the relocation constructor, as it is able to elide the destructor call of `src` ;

- The `src` parameter may be provided by the non-eliding version. This `src` may be a copy of the actual source object that was originally passed to the assignment operator in user code. Whether a copy is made depends on whether the non-eliding version is in charge of destroying its parameter. If not (`src` is an *unowned parameter* with regards to the non-eliding operator) then a copy is made, and the destructor of the source object is called at call-site, after the assignment operator returns.

This approach is likely the most efficient one, although it is not exception-safe. We recommend either the `noexcept` specification or the static assertions to be part of the implementation to make sure of that.

If `T` is trivially relocatable, then the operator is as optimal as we would like, as it merely translates into a destructor call and a memcpy call.

Union trick

If for some reason, the implementation needs to prevent the destructor call on the source object, it is still possible to perform the "union trick":

```
T& operator=(T src)
{
    union { T tmp } = { .tmp = reloc src; };
    /* do some stuff with tmp (like calling std::destroy_relocate),
     * knowing its destructor will not be called by the language */
    return *this;
}
```

5.3.4. Invocation

```
T x, y;
x = reloc y;
```

Every call to the relocation assignment operator follows normal rules.

If the call site detects that an eliding version of the operator is available (either because the eliding happened at declaration level, or because it happened at definition level and the call site is in the same translation unit as the definition, or through link-time optimization), then which version of the operator is called is implementation-defined.

The nominal case is to call the non-eliding version. The implementation is allowed to call the eliding version instead, as long as it can elide the call to the destructor on the source object.

5.4. Overload resolution

> NOTE: Compare [P2665R0] "Allow calling overload sets containing T, const T&".

The current overload resolution rules are not suitable for relocation by prvalue.

Indeed, consider the following scenario:

```
void bar(T&&);
void bar(T);

void foo(T val)
{
    bar(reloc val); /* ambiguous call using today's rules */
}
```

Hence we propose a change in the overload resolution rules to prefer passing by value for prvalue arguments.

Specifically, we would amend [over.ics.rank]/3.2.3 to read:

- neither of S1 and S2 bind a reference to an implicit object parameter of a non-static member function declared without a ref-qualifier, and either:
  - S1 binds an lvalue reference to an lvalue, and S2 does not, or:
  - S1 binds an rvalue reference to an xvalue, and S2 does not, or:
  - S1 does not bind a reference, and S2 binds a reference to a prvalue, or:
  - S1 binds an rvalue reference to a prvalue, and S2 binds an lvalue reference [Example:

```
int i;
int f1();
int&& f2();
...
int g2(const int&);
int g2(int);
int g2(int&&);
int j2 = g2(i); // calls g2(const int&)
int k2 = g2(f1()); // calls g2(int)
int l2 = g2(f2()); // calls g2(int&&)
...
```

- end example]

§ 5.5. Structured relocation

§ 5.5.1. Discussion

`auto [x, y] = foo(); sink(reloc y);`, if ill-formed given the rules we established for `reloc`. `x` and `y` are not complete objects but aliases to some anonymous object the language creates behind the scene.

The proposal aims to provide support for relocate-only types. This support would be partial, if not impractical, without allowing some form of relocation from a structured binding. This is motivated by:

- The need to make APIs that support relocate-only types. How would we write an API to extract an item at an arbitrary position from a vector? We propose the following API: `std::pair<T, iterator> vector<T>::erase(std::relocate_t, const_iterator);` (returns relocated vector element and next valid iterator) as it is consistent with other vector APIs and complies with the core guidelines. Then, what can users do with the returned object as it lies in a pair, and that it is forbidden to relocate a subobject? The return value is unusable for relocate-only types, unless we provide some support for it.
- In our experience, most C++ developers believe that a structured binding is a complete, separate object, and not a name alias to some subobject. As such it would feel unnatural for them if they cannot relocate from a structured binding.

§ 5.5.2. Structured relocation declaration

A structured relocation declaration is syntactically identical to a structured binding, with the exception that no ref-qualifiers are allowed after the `auto` type specifier.

```
T foo();
T const& bar();
T foobar();

// [...]
```

```
auto [x, y] = foo(); // matches structured relocation declaration
auto const [w, z] = bar(); // matches structured relocation declaration
auto&& [a, b] = foobar(); // structured bindings will be used
```

The structured relocation declaration further requires that the type of the expression that is used to initialize it supports object decomposition. If not, then the declaration is simply a structured bindings declaration and will follow structured bindings rules.

A structured relocation introduces a new complete object for each identifier declared in the brackets `[]`. In other words, the new identifiers are not aliases like in structured bindings, but actual complete objects. As such, they can then be relocated like any other.

5.5.3. Object decomposition

As there are three binding protocols for structured bindings, there are two "object decomposition" protocols for structured relocation. If none of those two protocols matches, then the declaration is not a structured relocation declaration.

First, *get_all* protocol is tested, and then the *data members* protocol.

In what follows, let `E` be the type of the initializer expression (the type of the expression used to initialize the structured relocation).

- If `E` is ref-qualified, then let `S` be the same as `E`, but deprived of its ref-qualifiers. If one of the two protocols applies, then an anonymous object of type `S` is constructed from the initializer, using the appropriate constructor. This anonymous object will be considered as source object ;
- Otherwise (`E` is a prvalue), then let `S` be the same as `E`. The initializer expression will be used as source object.

5.5.3.1. data members protocol

The *data members* protocol is quite similar to that of structured bindings. For this protocol to apply, all the following conditions must be satisfied:

- every non-static data member of `S` must be a direct member of `S` or of the same base class of `S` ;
- the number of identifiers must equal the number of non-static data members ;
- `S` may not have an anonymous union member ;
- *specific to structured relocation:* every base class, between `S` and the base class the data members are found in, does not have a user-defined destructor.

If this protocol applies, then the *i*-th identifier is constructed by relocation or synthesized relocation (move constructor ignoring cv-qualifiers, followed by destructor call) using the *i*-th data member of the source object.

5.5.3.2. get_all protocol

The function `get_all(S)` is looked-up using ADL-lookup. If there is no match, then this protocol does not apply.

If there is a match, then this function is called. The returned type is again tested against the two protocols. If *get_all* matches for the returned type, then we reapply it again, so on and so forth, until *get_all* doesn't match and only *data members* does.

This follows the same recursive logic as `operator->()`. We recursively call `get_all` as long as the *get_all* protocol applies. When the recursion ends, we end up with a type which matches the *data member* protocol.

The program is ill-formed if `T` matches the *get_all* protocol but the return type of `get_all(T)` matches none of the two protocols.

5.5.3.2.1. std::tuple and std::array are implementation-defined

`std::pair`, `std::tuple`, and `std::array` shall provide their own implementation of `get_all`. The return type is implementation-defined (may rely on compiler magic).

This allows us to write things like:

```
void bar(T);
void foo(std::vector<T>& v)
{
    /* erase(std::relocate_t) removes a vector element at given iterator,
     * returns a pair with next valid iterator and relocated vector element. */
    auto [val, it] = v.erase(std::relocate, v.begin() + 1); /* calls get_all behind
        the scene. */
    bar(reloc val); /* can call reloc on val as it is not a structured binding */
}
```

This code works even if `T` is relocate-only.

5.5.3.2.2. Possible get_all implementations

Most of the time, `get_all` functions will want to be § 5.6 Decomposing functions. A `get_all` implementation as a decomposing function is provided in the mentioned section.

Thanks to `std::tuple`'s `get_all` we can easily write a `get_all` implementation for a custom class:

```
class MyType
{
public:
    MyType();
    MyType(MyType);

    // Possible implementation:
    auto get_all(this MyType self)
    {
        return std::tuple{std::relocate, std::move(self._name), self._flag,
            !self._nodes.empty()};
    }

private:
    std::string _name;
    bool _flag;
    std::vector<Node*> _nodes;

};
```

The implementation relies on the proposed new constructor for `std::tuple`:

```
template <class... Tp>
tuple::tuple(std::relocate_t, Tp);
```

which captures the tuple elements by value and relocates them inside the tuple. `std::relocate_t` is just a tag type used for overload disambiguation.

Then, in following snippet:

```
MyType tp;
auto [name, flag, nodes] = reloc tp;
// equivalent to: auto [name, flag, nodes] = get_all(get_all(reloc tp));
```

`MyType`'s `get_all` returns a tuple. `get_all` is defined for tuples as well, so it is called again. The second return type won't have a `get_all` defined, hence the recursion stops and the *data member* protocol is used.

§ 5.6. Decomposing functions

We established that `reloc` can only be used to relocate function-local variables. Thus it follows that `reloc` cannot be used on data-members, which is impractical when we consider § 5.5 Structured relocation and user-defined `get_all` functions:

```
struct MyClass
{
    gsl::non_null<std::unique_ptr<int>> _reloc_only;

    auto get_all(this MyClass self)
    {
        return reloc self._reloc_only; // ill-formed, cannot relocate data members
    }
}
```

We introduce a special kind of member functions, called *decomposing functions*, to enable relocation of data-members. A decomposing function is a member function that takes an explicit `this` parameter by value and whose name is `reloc`:

```
auto get_all(this T reloc)
```

§ 5.6.1. Declaration

The decomposing trait of a function may not appear in the declaration if the parameter name is omitted or different than `reloc`. The decomposing trait is an implementation detail and it does not need to appear in the declaration.

A decomposing function does not bear any constraints on the number and types of parameters, except for its first parameter. The first parameter must be an explicit object parameter passed by value, possibly cv-qualified.

Further constraints apply to the type of the first parameter. For instance the type must be same as, or derive from the class-type the function is a member of. However those constraints are only checked in the context of the definition, not the declaration. See also here).

§ 5.6.2. Definition

A decomposing function decomposes its `this` parameter into subobjects before the function body is entered. It is subject to the following rules:

- The explicit `this` object is inaccessible in the function body ;
- Upon calling the function, before the function body is entered, `*this` is decomposed into individual objects for each direct base and non-static data member. This decomposition is a no-op: all subobjects of `*this` merely get considered as distinct complete objects. In particular the decomposition does not odr-use any of the copy, move or relocation constructor of the subobjects.
- Although the lifetime of `*this` ends once decomposed, its destructor is never called ;

- The direct base objects have no identifiers, and can only be accessed once through the `reloc Base` expression (`Base` being a direct base of the class-type). In particular, base data members are inaccessible in the function body ;
- The non-static data member objects are identified by their name in the class declaration and can be accessed normally. If disambiguation is necessary then the data members can be accessed with the syntax `T::datamember`, `T` being the type of the `this` parameter, and `datamember` being the name of the data member.

Special considerations are taken for overlapping subobjects:

- **EBO bases and `[[no_unique_address]]` members:** Obey the above rules.
- **Anonymous unions:** Obey the above rules. `reloc` rules pertaining to function-local anonymous union members apply.
- **Virtual bases (and bases with virtual bases):** are unrelocatable. They cannot be accessed by the `reloc Base` expression (ill-formed).

Since subobjects have become complete objects, they can be relocated like any function-local variable; otherwise, their destructor is called on exit from function scope.

§ *5.6.2.1. Examples*

A possible get_all implementation with a relocate-only type

```
struct Addr
{
    gsl::non_null<std::unique_ptr<int>> _addr;

    auto get_all(this Addr reloc) { return _addr; }
};
struct T : Addr
{
    std::vector<int> _data;

    auto get_all(this T reloc)
    {
        auto const [addr] = reloc Addr;
        return std::pair{reloc addr, reloc _data};
    }
};
```

Curiously Recurring Template Pattern

```
template <class Derived>
struct ExtractAddr
{
    auto get_addr(this Derived reloc) { return Derived::_addr; }
};
struct ExtractAddr2
{
    template <class Derived>
    auto get_addr(this Derived reloc) { return Derived::_addr; }
};
struct A : ExtractAddr<A>
{
    gsl::non_null<std::unique_ptr<int>> _addr;
    // [...]
};
struct B : ExtractAddr2
{
```

```
    gsl::non_null<std::unique_ptr<int>> _addr;
    // [...]
};
```

Note that, in the above example, if we have a `DA` structure that inherits from `A` and a `DB` structure that inherits from `B`, then `DA{}.get_addr()` is well-formed while `DB{}.get_addr()` is not. Indeed `_addr` is not a direct member of `DB` (type deduced in `ExtractAddr2::get_addr`) and hence is not directly accessible this way.

A possible implementation of a putative unique_ptr::release (not proposed)

```
T* unique_ptr<T,D>::release(this unique_ptr reloc)
{
    /* the "owned" pointer is loose at this point because of the decomposition */
    return _ptr;
}
```

5.6.3. Ill-formed definition

If not forbidden, there are several ways where decomposition functions can be misused and lead to problems. In this section we will see those cases and they will serve as motivation to the ill-formedness criteria.

5.6.3.1. Aberrations

As mentioned in [P0847R6], in the pathological cases section, code like this is possible:

```
struct A
{
    template <class T>
    auto foo(this T self);
};

auto func = &A::foo<std::unique_ptr<int>>;
func(std::make_unique<int>(0));
```

This remains acceptable in the context of P0847R6, but may lead to dangerous code with decomposing functions. We don't want the following code to be valid:

```
struct Sink
{
    template <class T>
    void sink(this T reloc) {} /* prevent dtor call,
        although each subobject is destroyed individually */
};

auto func = &Sink::sink<std::unique_ptr<int>>;
func(std::make_unique<int>(0)); // leak
```

We don't want users to decompose types that were never designed to support decomposition. Ideally, a decomposing function should be defined for the same source object type it is declared in (i.e. the explicit `this` parameter type should match the class-type which the function is a member of). However we feel this is too restrictive, especially when considering template code and the CRTP pattern.

Hence we require the source object type to be the same as or to inherit from the class-type the decomposing function is a member of, or else the definition of the decomposing function is ill-formed.

5.6.3.2. Accessibility

Let's imagine for a moment that the source object has inaccessible subobjects from the context of the decomposing function.

Upon entering the function, all subobjects are promoted to complete objects. As we don't want to break accessibility rules, we may forbid access to the objects that previously were inaccessible subobjects.

However this is not necessary. If they are never accessed, then it means that their destructor will be called when the function exits. Calling the destructor requires them to be accessible in the first place.

Hence, we further mandate that all subobjects of the source object be accessible from the context of the decomposing function.

Note that this condition is fulfilled in the nominal case, where the decomposing function is merely a member function of the source object, and hence is granted access to all of its subobjects.

5.6.3.3. ABI requirements

The decomposing trait does not appear in the function declaration, and as such a decomposing function does not have any specific ABI. In particular it shares the same ABI as any explicit `this` member function where the `this` paramater is passed by value.

Some ABIs may not natively be compatible with decomposing functions, especially if the ABI specifies that the destruction of the source object happens at the call site and not in the decomposing function.

To support such cases, we need to allow implementors to silently and transparently make a temporary copy of the source object, and pass that temporary copy to the decomposing function (see § 5.7.3 Decomposing functions).

This requires the source object to have at least one of the copy, move or relocation constructor.

Note that, for regular functions, function parameters passed by value can only be passed to `reloc` if they provide at least one of the three constructors. This requirement follows the same spirit.

5.6.3.4. Ill-formedness criteria

Now that we have seen motivating cases, let us summarize the ill-formedness criteria for a decomposing function. Let a decomposing function be defined in a class-type `T`, which takes an explicit `this` parameter (the source object) of type `S`. The definition is ill-formed if:

- the explicit `this` parameter is not passed by value and is named `reloc` ;
- or the decomposing function is not a member function ;
- or `S` has any inaccessible subobjects with regards to the decomposing function ;
- or `S` is not the same as `T`, and does not derive from `T` ;
- or `S` has no accessible relocation constructor, no accessible move constructor and no accessible copy constructor (none of the three).

5.6.4. Decomposing lambda

A decomposing lambda is a lambda function that takes an explicit `this` parameter by value and whose name is `reloc`.

As for decomposing functions, a decomposing lambda is ill-formed if:

- the type of the `this` parameter is not passed by value ;
- or the parameter type is not the same as, or does not derive from the type of the lambda (see § 5.6.3.1 Aberrations) ;

- or the parameter type has no relocation, move and copy constructors (none of the three).

A decomposing lambda acts as a decomposing function: when called, before the lambda body is entered, all the lambda captures are considered as distinct complete objects. As such, the lambda captures can be relocated.

```
void foo(reloc_only_t);

reloc_only_t var;
auto func = [v = reloc var](this auto reloc)
{
    foo(reloc v);
};
(reloc func)(); // well-formed
```

§ 5.7. ABI changes

As noted above (§ 3.3 Unowned parameter), some platforms have a *caller-destroy* ABI where the *calling* function expects to destroy nontrivial parameters passed by value. This poses a problem for functions that wish to relocate from such parameters, and a potential ABI break.

§ 5.7.1. relocate-only function parameters

We propose the following requirement on functions: if a function takes a parameter by value, whose type is relocate-only, then the function is responsible for the destruction of that parameter.

A relocate-only type is a type that declares a non-deleted relocation constructor, the move and copy constructors being not declared, or declared as deleted. This requirement is essential to fully support relocate-only types in the language.

This requirement *might* introduce an ABI break. As of today, there are no relocate-only types, so no ABI should break. In the proposed library changes, we do not make any existing type relocate-only, especially for that concern. However we do add a relocation constructor on many classes, alongside their existing copy and move constructors. In doing so, some of them may become relocate-only, should their copy and move constructors be deleted (for instance `std::optional<T>` with `T` being relocate-only).

One example is a function with signature: `void foo(gsl::non_null<std::unique_ptr<int>>);`. We propose to add a relocation constructor to `unique_ptr`, and GSL developers will likely add a relocation constructor too. That makes `gsl::non_null<std::unique_ptr<int>>` relocate-only, while it wasn't before, and may cause a potential ABI break.

There is zero value of passing a `gsl::non_null<std::unique_ptr>` by value to a function today, so we doubt anyone would write such a function. However those functions might theoritically exist, and might have an ABI change.

Also, library vendors are encouraged to migrate to an ABI where any function that takes non-trivial parameters by value are responsible for their destruction. Then, the function definition can make the most of `reloc`. This is not required by the proposal.

We believe it's up to the implementation to choose what they want to do with their ABI:

- *full break*, use callee-destroy or equivalent for all non-trivial relocatable types passed by value (for those who don't care about ABI) ;
- *break with opt-out*: a relocation constructor attribute to opt-out of the ABI break on functions where it is passed be value. This solution should also provide propagation mechanisms suitable for composition (these could be standardized at a later date) ;
- *likely no break, but opt-in*: an improved `[[trivial_abi]]` that actually checks that the type is trivially relocatable ;
- *likely no break*: use callee-destroy only for relocate-only types ;
- and those that are callee-destroy already don't need to do anything!

In all cases the following mitigation and migration techniques could be employed:

- functions that have an ABI change could be mangled differently. This makes ABI breakage detectable ;
- for such functions, up to two symbols are emitted, where the old symbol is emitted only if the function does not in actual fact relocate from its parameters, in which case the new symbol is emitted, and its implementation forwards to the old and then destructs its relocatable parameters on exit ;

5.7.2. prvalue assignment operator

As mentioned above, if the class-type is relocate-only, then it may have an impact on existing prvalue-assignment operators (like it does to any function). However this change is purely opt-in. If there is an existing prvalue-assignment operator in a class, then it will prevent the implicit declaration of the relocation constructor, which will in turn prevent from the potential ABI break.

Also, the relocation assignment operator may be aliased. If aliasing occurs, then the ABI does not break as aliasing happens only on a new hidden function.

The only scenario where the ABI might break is where:

- aliasing happened on declaration level ;
- code was compiled against it, and especially generated code that makes direct calls to the aliased version ;
- the class changes, the aliasing only happens at definition level, or does not happen at all.

This may introduce an ABI break, detectable at link-time (aliased symbols missing):

- if the aliasing now happens at definition level, but the aliased operator symbol remains visible nonetheless, then no ABI breaks are introduced ;
- otherwise the ABI break happens, but remains detectable at link-time.

5.7.3. Decomposing functions

A decomposing function has an extra ABI requirement: the function must be in charge of the lifetime of its explicit `this` parameter. How this requirement is satisfied is up to compiler vendors.

Some tricks are possible for ABIs that do not natively meet this requirement. An attractive solution is to proceed as with the relocation assignment operator; when a decomposing function is defined, two functions are emitted:

- an eliding one: this function takes the explicit `this` parameter as if by reference and is in charge of its lifetime. Its body is the body of the decomposing function provided by the user.
- a non-eliding one:
  - If the explicit `this` parameter is not an *unowned parameter*, then it simply calls the eliding one, passing the `this` parameter object as if by reference.
  - Otherwise it makes a temporary copy of the source object, and calls the eliding one, passing the copy as if by reference. Upon function exit, the destructor of the copy is not called as its lifetime already ended when it was passed the eliding function.
  - In all cases, it forwards as return value whatever the decomposing function returns.

The attentive reader will have noticed that a program with a decomposing function is ill-formed if the class-type does not provide a relocation, move or copy constructor. Hence, the non-eliding function is able to perform a copy if necessary without adding extra requirements on the class-type.

6. Proposed library changes

6.1. Memory header

6.1.1. std::construct_at

We propose to add the following overload to `std::construct_at`:

```
template<class T>
constexpr T* construct_at( T* p, T src );
```

Which would be equivalent to `::new (p) T{reloc src}`, except that it may be used in constant expression evaluations.

> NOTE: this overload would be unnecessary if the § 7.3.1 More perfect forwarding direction were to be adopted; instead the existing signature should be altered to use the `decltype(auto)` placeholder.

6.1.2. std::destroy_relocate

We propose to add the following function in the `std` namespace in the `memory` header to perform relocation through a pointer:

```
template <class T>
T destroy_relocate(T* src);
```

The function constructs a new object by calling either the relocation constructor, the move constructor, or the copy constructor (in that order of preference), using `*src` as parameter while ignoring its cv-qualifiers:

- If the move or copy constructor is called then the destructor of `*src` is called afterwards ;
- Likewise, if the move or copy constructor throws, then the destructor of `*src` is called as well.

The function returns the constructed value. Its definition is implementation-defined.

This function is intended to be used by library authors, to enable relocation from a memory address. For instance, extracting a value out of an optional just becomes:

```
T optional<T>::extract()
{
    _has_value = false;
    return std::destroy_relocate(_value_addr());
    // _value_addr() being a private function returning the address of the owned value
}
```

This function is not intended to be used on local objects:

```
void foo()
{
    const T val;
    bar(std::destroy_relocate(&val)); /* BAD, val destructor is called at the
        end of its scope while it is already destructed!*/
}
```

This what motivates the name of the function. Although relocation is always a destructive operation, the name serves as a reminder to the developers.

6.1.3. std::uninitialized_relocate

We propose to introduce the following new functions in the `std` namespace in the `memory` header:

```
template<class InputIt, class ForwardIt>
ForwardIt uninitialized_relocate(InputIt first, InputIt last, ForwardIt d_first);

template<class ExecutionPolicy, class InputIt, class ForwardIt>
ForwardIt uninitialized_relocate(ExecutionPolicy&& policy, InputIt first, InputIt last,
        ForwardIt d_first) ;

template<class InputIt, class Size, class ForwardIt>
pair<InputIt, ForwardIt> uninitialized_relocate_n(InputIt first, Size count,
        ForwardIt d_first) ;

template<class ExecutionPolicy, class InputIt, class Size, class ForwardIt>
pair<InputIt, ForwardIt> uninitialized_relocate_n(
        ExecutionPolicy&& policy, InputIt first, Size count, ForwardIt d_first);
```

Those relocate elements from the range `[first, last)` (or the first `count` elements from `first`) to an uninitialized memory area beginning at `d_first`. Elements in the source range will be destructed at the end of the function (even if an exception is thrown).

Returns:

- `uninitialized_relocate`: an iterator to the element past the last element relocated;
- `uninitialized_relocate_n`: a pair whose first element is an iterator to the element past the last element relocated in the source range, and whose second element is an iterator to the element past the last element relocated in the destination range.

If the type to relocate is trivially relocatable and both iterator types are contiguous, then both functions can be implemented as single `memcpy` call over the entire source range. Otherwise relocation happens element-wise, as if by calling `std::destroy_relocate` on each element.

If an exception is thrown by `std::destroy_relocate`, then the destructor of all remaining elements in the source range is called, as well as the destructor of all constructed objects in the output iterator.

6.2. Utility header

6.2.1. std::relocate

We propose to add the following tag type in the `std` namespace in the `utility` header (mainly useful with templates):

```
namespace std
{
// tag type to indicate that the parameters are passed by value
struct relocate_t {};
inline constexpr relocate_t relocate = {};
}
```

> NOTE: this facility would be unnecessary if the § 7.3.1 More perfect forwarding direction were to be adopted; instead the existing signatures should be altered to use the `decltype(auto)` placeholder.

6.3. Bring relocate-only type support to the STL

6.3.1. std::pair and std::tuple

We propose to add a default relocation constructor and a default relocation assignment operator to `std::pair` and `std::tuple`.

We also porpose to add the following functions:

```
template <class T1, class T2>
pair<T1, T2>::pair(std::relocate_t, T1, T2); // constructs by relocation

template <class... Types>
tuple<Types...>::tuple(std::relocate_t, Types...); // constructs by relocation

template <class... Types>
template <class U1, class U2>
tuple<Types...>::tuple(std::pair<U1, U2>); // constructs by relocation
```

Note that we do not introduce extra template parameters for type arguments, as relocation can only happen from matching types.

6.3.2. std::optional

We propose to add the following functions to `std::optional`:

```
// relocation constructor
template <class T>
optional<T>::optional(optional);

// relocation assignment operator
template <class T>
optional& optional<T>::operator=(optional);

// Converting constructor
template <class T>
optional<T>::optional(T val);

// Converting assignment
template <class T>
optional& optional<T>::operator=(T val);

/**
 * \brief Extracts the contained value from the optional
 *
 * The returned value is relocated from the contained value.
 *
 * After this call the optional no longer contains any value.
 *
 * \throws std::bad_optional_access if the optional did not contain any value.
 */
template <class T>
T optional<T>::extract();
```

6.3.3. std::variant

```
// relocation constructor
template <class... Types>
variant<Types...>::variant(variant);

// relocation assignment operator
template <class... Types>
variant& variant<Types...>::operator=(variant);

// Converting constructor
template <class... Types>
template <class T>
constexpr variant<Types...>::variant(T val);

// Converting assignment
template <class... Types>
template <class T>
variant& variant<Types...>::operator=(T);
```

6.3.4. std::any

```
// relocation constructor
any::any(any);

// relocation assignment operator
any& any::operator=(any);

// Converting constructor
template <class T>
any::any(T);

// Converting assignment
template <class T>
any& any::operator=(T);
```

6.4. Containers

All containers must provide a relocation constructor and a relocation assignment operator.

Also, in order to fully support relocate-only types, containers should provide a way to insert and remove elements by relocation.

6.4.1. Insertion overloads

Existing APIs cannot fulfill this need. They all take the element to insert as a reference parameter, while relocation requires to pass elements by value.

As such we suggest adding overloads to all insertion functions, where the element to insert is passed by value.

6.4.2. Extracting functions

The STL does not provide any function to erase an element from a container and return it as return value.

Consider a container of relocate-only types. If an element of that container is to be "moved out" of it, it could only happen through relocation as it is the only operation supported by the type. Hence the relocated element must necessarily be simultaneously erased from the container as its lifetime ended.

This is why we propose to add various "value-extracting" functions to existing containers, which erase an element and return it. Those functions are overloads over existing `pop*()` and `erase()` functions, but they take the `std::relocate_t` tag type as first parameter. The return value will be constructed by relocation (likely thanks to `std::destroy_relocate`).

All extracting functions will operate the same way. First, the return value is constructed as if by `std::destroy_relocate` from the container element. Second, the container adjusts its size and memory to effectively erase the contained element from its internal data structures.

If an exception is emitted during the first step, then the container proceeds to erase its element nonetheless (as if in the second step) and then propagates the exception. If an exception is emitted during the second step (regardless of whether the second step was triggered normally of by an exception caught during the first step), then `std::terminate` is called.

6.4.3. relocate_out

We further propose to add a `relocate_out` function to some containers. `relocate_out` takes three iterators as parameters. The first two iterators belong to the container and define the range to relocate. The last parameter is an output iterator where the relocated elements will be constructed. This is similar the `erase` functions that take a range of elements, except that an extra output iterator is provided. It returns a pair of iterators, whose first iterator is the next valid iterator in the container (similar to what `erase` returns), and whose second iterator is the output iterator, past the last element copied (similar to what `std::copy` returns).

`relocate_out` is proposed to improve support of relocate-only types. Without this, it would not be possible to move a range of relocate-only elements from one container to another, without writing complex and inefficient loops calling some value-extracting function at each iteration.

Note that there is less need for a `relocate_in` function as the `std::insert_iterator` family will have an overload to enable relocation.

`relocate_out` proceeds as follows:

1. relocates the elements within range to the output iterator. The elements within range inside the container are then in a *destructed state*.
2. The destructed elements are removed from the container. How this is achieved depends on the container. For instance `std::vector` may call `std::uninitialized_relocate` to move the trailing part of the container in the destructed range, and simply reduce its size.

If an exception leaks through the first step, then the second step is run to erase from the container all the elements that are in a *destructed state* (i.e. those which got succesfully relocated plus the one responsible for the exception) , and the exception is propagated. If an exception is emitted during the second step (regardless of whether the second step was triggered normally of by an exception caught in the first step), then `std::terminate` is called.

6.4.4. std::vector

```
// pushes a value by relocation
template <class T, class Alloc>
constexpr void vector<T, Alloc>::push_back(T value);

// inserts a value by relocation
template <class T, class Alloc>
iterator vector<T, Alloc>::insert(const_iterator pos, T value);

// removes the last item from the vector and returns it
```

```
template <class T, class Alloc>
T vector<T, Alloc>::pop_back(std::relocate_t);

// removes the item from the vector and returns it with the next valid iterator
template <class T, class Alloc>
std::pair<T, iterator> vector<T, Alloc>::erase(std::relocate_t, const_iterator pos);

// relocates items in [from, to[ into out,
// as if by doing iteratively: *out++ = std::destroy_relocate(&*src++);
// items within range are removed from *this.
template <class T, class Alloc>
template <class OutputIterator>
std::pair<iterator, OutputIterator> vector<T, Alloc>::relocate_out(
        const_iterator from, const_iterator to, OutputIterator out);
```

6.4.5. std::deque

```
// pushes a value by relocation
template <class T, class Alloc>
constexpr void deque<T, Alloc>::push_front(T value);
template <class T, class Alloc>
constexpr void deque<T, Alloc>::push_back(T value);

// inserts a value by relocation
template <class T, class Alloc>
iterator deque<T, Alloc>::insert(const_iterator pos, T value);

// removes the last item from the queue and returns it
template <class T, class Alloc>
T deque<T, Alloc>::pop_back(std::relocate_t);
// removes the first item from the queue and returns it
template <class T, class Alloc>
T deque<T, Alloc>::pop_front(std::relocate_t);
// removes the item from the queue and returns it with the next valid iterator
template <class T, class Alloc>
std::pair<T, iterator> deque<T, Alloc>::erase(std::relocate_t, const_iterator pos);

// relocates items in [from, to[ into out.
// items within range are removed from *this.
template <class T, class Alloc>
template <class OutputIterator>
std::pair<iterator, OutputIterator> deque<T, Alloc>::relocate_out(
        const_iterator from, const_iterator to, OutputIterator out);
```

6.4.6. std::list

```
// pushes a value by relocation
template <class T, class Alloc>
void list<T, Alloc>::push_front(T value);
template <class T, class Alloc>
void list<T, Alloc>::push_back(T value);

// inserts a value by relocation
template <class T, class Alloc>
iterator list<T, Alloc>::insert(const_iterator pos, T value);
```

```
// removes the last item from the list and returns it
template <class T, class Alloc>
T list<T, Alloc>::pop_back(std::relocate_t);
// removes the first item from the list and returns it
template <class T, class Alloc>
T list<T, Alloc>::pop_front(std::relocate_t);
// removes the item from the list and returns it with the next valid iterator
template <class T, class Alloc>
std::pair<T, iterator> list<T, Alloc>::erase(std::relocate_t, const_iterator pos);

// relocates items in [from, to[ into out.
// items within range are removed from *this.
template <class T, class Alloc>
template <class OutputIterator>
std::pair<iterator, OutputIterator> list<T, Alloc>::relocate_out(
        const_iterator from, const_iterator to, OutputIterator out);
```

6.4.7. std::forward_list

```
// inserts a value by relocation
template <class T, class Alloc>
iterator forward_list<T, Alloc>::insert_after(const_iterator pos,
        T value);
template <class T, class Alloc>
void forward_list<T, Alloc>::push_front(T value);

// removes the first item from the list and returns it
template <class T, class Alloc>
T forward_list<T, Alloc>::pop_front(std::relocate_t);
// removes the item after pos from the list and returns it with the iterator following pos
template <class T, class Alloc>
std::pair<T, iterator> forward_list<T, Alloc>::erase_after(std::relocate_t, const_iterator

// relocates items in ]from, to[ into out.
// items within range are removed from *this.
template <class T, class Alloc>
template <class OutputIterator>
OutputIterator forward_list<T, Alloc>::relocate_after(
        const_iterator from, const_iterator to, OutputIterator out);
```

6.4.8. set and map containers

```
// std::set, std::multiset, std::map, std::multimap,
// std::flat_set, std::flat_multiset, std::flat_map, std::flat_multimap,
// std::unordered_set, std::unordered_multiset, std::unordered_map
// and std::unordered_multimap, all aliased as 'map':
std::pair<iterator, bool> map::insert(value_type value);
iterator map::insert(const_iterator hint, value_type value);

// extract the stored value from the container
std::pair<value_type, iterator> map::erase(std::relocate_t, const_iterator position);
```

6.4.9. queues

```
// for std::stack, std::queue, std::priority_queue, aliased queue below:
void queue::push(T value);

// removes the next element from the queue, calling the appropriate pop function
// on the underlying container with std::relocate as parameter
T queue::pop(std::relocate_t);
```

6.4.10. Iterator library

We propose to add the following overloads:

```
template <class Container>
back_insert_iterator<Container>& operator=( typename Container::value_type value );

template <class Container>
front_insert_iterator<Container>& operator=( typename Container::value_type value );

template <class Container>
insert_iterator<Container>& operator=( typename Container::value_type value );
```

6.4.11. Other STL classes

We propose to add a relocation constructor and a relocation assignment operator to all the following classes:

- **String library**: `std::basic_string` ;
- **Utility**: `std::function`, `std::reference_wrapper`, `std::shared_ptr`, `std::weak_ptr`, `std::unique_ptr` ;
- **Regular expression**: `std::basic_regex`, `std::match_results` ;
- **Thread support**: `std::thread`, `std::jthread`, `std::unique_lock`, `std::promise`, `std::future`, `std::shared_future`, `std::packaged_task` ;
- **Filesystem**: `std::filesystem::path`.

7. Discussions

7.1. Why a new keyword?

Alternatively, a new series of symbols could be used instead of introducing a new keyword, like: `<~< obj` or `&< obj` in place of `reloc obj`. However, we feel like `reloc obj` better conveys the intent, and has better readibility.

The introduction of a new keyword may always break existing codebases. We had a look at several well-known open source C++ projects to analyse what volume of code would break if `reloc` were a keyword.

For each of the following repositories, we searched for the `reloc` string, at word boundaries, with case-sensitivity, in all C++ source files and headers (`*.cc`, `*.cpp`, `*.cxx`, `*.h`, `*.hpp`, `*.hh`). We manually discarded matches that were not code (comments or strings). And we put that in perspective with the total number of files, lines and words of the repository.

- Qt: 0 hits; files: 7,586 ; lines: 2,794,607 ; words: 98,635,622; commit: 040b4a4b21b3
- boost (with all submodules): 0 hits; files: 23,726 ; lines: 4,133,844 ; words: 180,808,943; commit: 86733163a3c6
- godot: 0 hits; files: 5,068 ; lines: 2,545,299 ; words: 99,389,743; commit: b6e06038f8a3

- abseil-cpp: 0 hits; files: 766 ; lines: 247,441 ; words: 9,028,820; commit: de6fca2110e7
- folly: 0 hits; files: 1,861 ; lines: 532,918 ; words: 16,669,085; commit: cde9d22e8614
- llvm-project: **124 hits in 11 files** (`reloc` only used as local variable or data-member, counting all uses); files: 39,048 ; lines: 9,760,587 ; words: 385,429,611; commit: 9816c1912d56
- gcc: **244 hits in 31 files** (`reloc` only used as local variable or data-member, counting all uses); files: 15,337 ; lines: 4,616,875 ; words: 146,146,684; commit: ee6f262b87fe
- rapidjson: 0 hits; files: 96 ; lines: 39,828 ; words: 1,492,060; commit: a98e99992bd6
- googletest: 0 hits; files: 155 ; lines: 85,703 ; words: 3,104,817; commit: 71140c3ca7a8
- yaml-cpp: 0 hits; files: 259 ; lines: 112,513 ; words: 3,784,676; commit: 1b50109f7bea
- flatbuffers: 0 hits; files: 175 ; lines: 98,163 ; words: 3,851,726; commit: e0d68bdda2f6
- MongoDB: **22 hits in 6 files** (`reloc` only used as local variable, counting all uses); files: 20,054 ; lines: 6,439,465 ; words: 265,329,429; commit: 73b7a22328c7
- OpenCV: 0 hits; files: 3,315 ; lines: 1,556,606 ; words: 58,339,686; commit: 9627ab9462a4
- electron: 0 hits; files: 698 ; lines: 99,717 ; words: 3,431,787; commit: 644243efd61b
- mold: 0 hits; files: 813 ; lines: 262,560 ; words: 9,992,769; commit: a45f97b47430
- ClickHouse: 0 hits; files: 5,566 ; lines: 1,128,735 ; words: 68,112,047; commit: d42d9f70c812;
- Dlib: 0 hits; files: 1,421 ; lines: 533,513 ; words: 19,080,728; commit: a12824d42584
- SFML: 0 hits; files: 532 ; lines: 168,787 ; words: 7,272,946; commit: 9bdf20781819
- Kodi: 0 hits; files: 4,360 ; lines: 1,008,255 ; words: 34,114,229; commit: b228c778668f
- Beast: 0 hits; files: 473 ; lines: 145,193 ; words: 4,768,152; commit: 334b9871bed6
- JSON for modern C++: 0 hits; files: 450 ; lines: 137,679 ; words: 5,210,982; commit: 4c6cde72e533
- IncludeOS: 0 hits; files: 841 ; lines: 107,582 ; words: 2,903,698; commit: 99b60c782161
- SerenityOS: **15 hits in 2 files** (`reloc` only used as local variable, counting all uses); files: 5,538 ; lines: 887,768 ; words: 31,766,641; commit: 97dde51a9b3f

Repository statistics are computed with the following command:

```
find -type f \( -name '*.h' -or -name '*.hh' -or -name '*.hpp' -or -name '*.cc' \
    -or -name '*.cpp' -or -name '*.cxx' \) -exec wc -l -c {} \; \
    | awk '{ f+=1 } { l += $1 } { w += $2 } END { print "files: ", f, "; lines: ", l, "; wo
```

As you can see, in the vast majority of cases, `reloc` is not used at all. The impact seems to be minimal, where only a few files might need to be fixed here and there. To smooth the transition, compilers may also warn that existing code will break as `reloc` will become a keyword in a next C++ version.

7.2. STL API changes

7.2.1. Why add overload to the value-extracting functions in STL containers?

In the original draft, all value-extracting functions were named `pilfer` (or `pilfer_back` and so on). However we felt it was better to reuse the function names that already exist.

We thought of simply changing the return value of existing functions: `vector::pop_back()` would return the erased value. C++17 already changed the return value of `vector::emplace_back`. It caused no ABI issue (the return type of template member function is part of the mangling), and no performance hit (as template functions are inlined, when the reference returned from `vector::emplace_back` is not used, then the instructions that return the reference are not emitted).

We could have done the same with `vector::pop_back()`. However, doing so would require that the type stored in the container to have one of the three constructors (relocation, move or copy). This is a requirement that doesn't exist today for node-based containers (`std::list`, `std::set` etc...). To state it otherwise, making an `T list<T>::pop_back()` will add unwanted constraints on what can be stored in a list.

In order to preserve a consistent API across all STL containers, we opted in favor of `pop(std::relocate_t)` overloads.

7.2.2. Why name the value-extracting function of optional extract?

As mentioned we introduce `T optional<T>::extract()` to relocate a value out of the optional.

Alternatively, this function could have been named `pop` if we want to stay consistent with what's done with containers.

However a container `pop()` function doesn't return anything, only the `pop(std::relocate_t)` overload does.

We prefer `extract()` over `pop(std::relocate_t)` for `std::optional`, as it better conveys the intent, is easier to write, and `std::optional` is not after all a container.

Another alternative would be to name all value-extracting functions with a new name, and have it in common with `std::optional` (like `pilfer()`), however we prefer to stick with existing API for containers.

7.3. Future directions

We removed some of the changes we initilialy had in mind, to keep for future extensions. This proposal aims to be the bare minimum to bring support for relocate-only types.

7.3.1. More perfect forwarding

Currently, "perfect forwarding" is built on top of *universal references*, requiring an understanding of reference-collapsing and the use of `std::forward`. The present proposal improves on this by incidentally replacing `std::forward` with `reloc`, but at the same time the situation is worsened by making relocate-only types viable; such types cannot be relocated when passed by universal reference.

Before	After	Future

`template`
`void fwd(Args&&... args)`
`{`
`  do_stuff(std::forward(args)...);`
`}`
	`template`
	`void fwd(Args&&... args)`
	`{`
	`  do_stuff(reloc args...);`
	`}`
		`void fwd(decltype(auto)...`
		`{`
		`  do_stuff(reloc args...`
		`}`

By allowing `decltype(auto)` as a *placeholder-type-specifier* in a *parameter-declaration* (i.e. relaxing [dcl.fct] paragraph 22) it would become possible to deduce each parameter to value, lvalue reference or rvalue reference according to whether the argument is of value category prvalue, lvalue or xvalue, and forward by relocation.

7.3.2. discarded reloc expression

Initilialy, discarded reloc expressions such as `reloc obj;` would simply translate to a call to the destructor of `obj`, while ensuring that the destructor won't be called again at the end of its scope.

However this is hardly possible at the moment because of all the different ABIs that exist. If `obj` is an *unowned parameter*, then the function cannot elide the destructor call of `obj` that will happen on the caller-side.

We wanted the well-formedness of the code above all else (i.e. `reloc obj;` could not be well-formed on some implementations and not in others). As such, in this proposal, `reloc obj;` is only well-formed if `obj` is relocatable, movable or copyable.

Hence, the best we can do if `obj` is an *unowned parameter*, is to move-construct a temporary, and destruct it right after, which will trigger the desired side-effects of the destructor (e.g. release a lock if `obj` is a `unique_lock`). The destructor of `obj` will still be called when the function returns, but will likely do nothing as the object will be in a moved-from state.

A future proposal could make `reloc obj;` to just call the destructor, regardless of whether `obj` is an *unowned parameter* and of its constructors, solving those ABI issues.

7.4. Will it make C++ easier?

Even though it does come with new rules, we argue that it mostly removes the moved-from state understanding problem, as well as used-after-move errors (if `reloc` is used instead of `std::move`).

References

Informative References

[D2839R1]
Brian Bi; Joshua Berne. *Nontrivial Relocation via a New owning reference Type*. June 2023. URL: https://isocpp.org/files/papers/D2839R1.html

[IIFE]
Bartlomiej Filipek. *IIFE for Complex Initialization - C++ Stories*. October 2016. URL: https://www.cppstories.com/2016/11/iife-for-complex-initialization/

[N4158]
Pablo Halpern. *Destructive Move*. October 2014. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4158.pdf

[P0023R0]
Denis Bider. *Relocator: Efficiently moving objects*. April 2016. URL: http://open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0023r0.pdf

[P0308R0]
Peter Dimov. *Valueless Variants Considered Harmful*. March 2016. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0308r0.html

[P0847R6]
Gašper Ažman; et al. *Deducing this*. January 2021. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r6.html

[P1029R3]
Niall Douglas. *move = bitcopies*. January 2020. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1029r3.pdf

[P1144R8]
Arthur O'Dwyer. *std::is_trivially_relocatable*. May 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p1144r8.html

[P2665R0]
Bengt Gustafsson. *Allow calling overload sets containing T, const T&*. October 2022. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2665r0.pdf